Aula 2: Engenharia de Dados para IA e Compliance

Prof. Ioannis Eleftheriou

LISTA DE PRESENÇA
TURMA 3

/ O que vimos até agora?

IOANNIS PEDRO ELEFTHERIOU

/ioannispedroeleftheriou

“Nenhuma boa ação sairá impune”

- Level: 30
- Classe: Especialista de IA e professor de Engenharia de Machine Learning e AI Scientist na FIAP
- Subclasse: MBA em Data Science pela FIAP e Bacharel em Game Design
- Analista Arcano: Mais de 10 anos de experiência com Crédito e Dados
- Empresas que atuei: HSBC, Bradesco, Vortx, Itaú, Grupo Boticário
- Criador de universos visuais e narrativas que tornam o conhecimento acessível e inspirador — da lousa ao código.

O que veremos hoje

RAG: Diferenciar um chat genérico de extração estruturada

Stack: Pydantic, Chroma DB, LangChain, PyPDF

Hands On: Rag

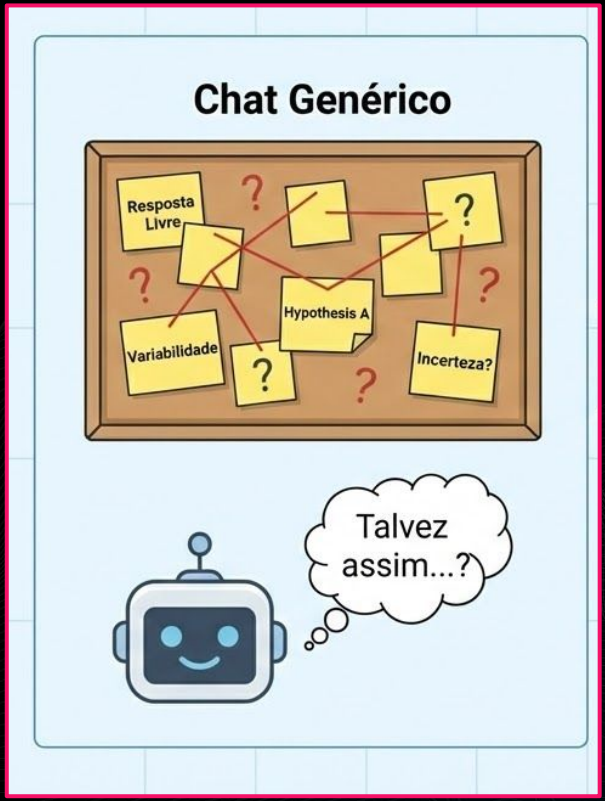
Exercício

/ RAG - Retrieval-Augmented Generation

/ Chat genérico vs Extração estruturada

Modelos de linguagem são gerativos, não determinísticos, então a mesma pergunta raramente vem exatamente igual duas vezes.

Em extração estruturada, ao invés de uma resposta em texto livre, buscamos que a IA produza dados em um formato padronizado e previsível.



Fonte da imgem: Autor, Nano Banana.

/ RAG
Retrieval-Augmented Generation

R - Retrieve: Encontra informações úteis

A - Augment: Adiciona elas ao conhecimento da IA

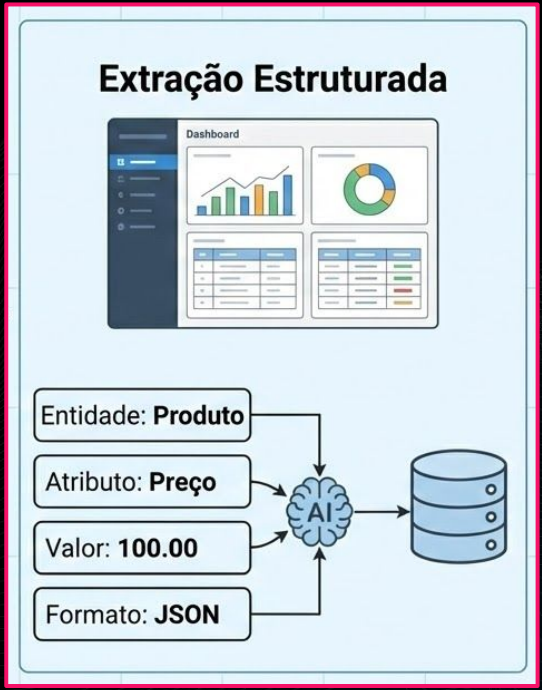
G - Generate: Gera uma resposta mais eficiente.

Fonte da imgem: Autor, Nano Banana.

/ RAG
Retrieval-Augmented Generation

RAG é dar ao modelo uma “cola” confiável: antes de gerar a resposta, ele busca informações relevantes em uma base de conhecimento fornecida, e usa isso de contexto

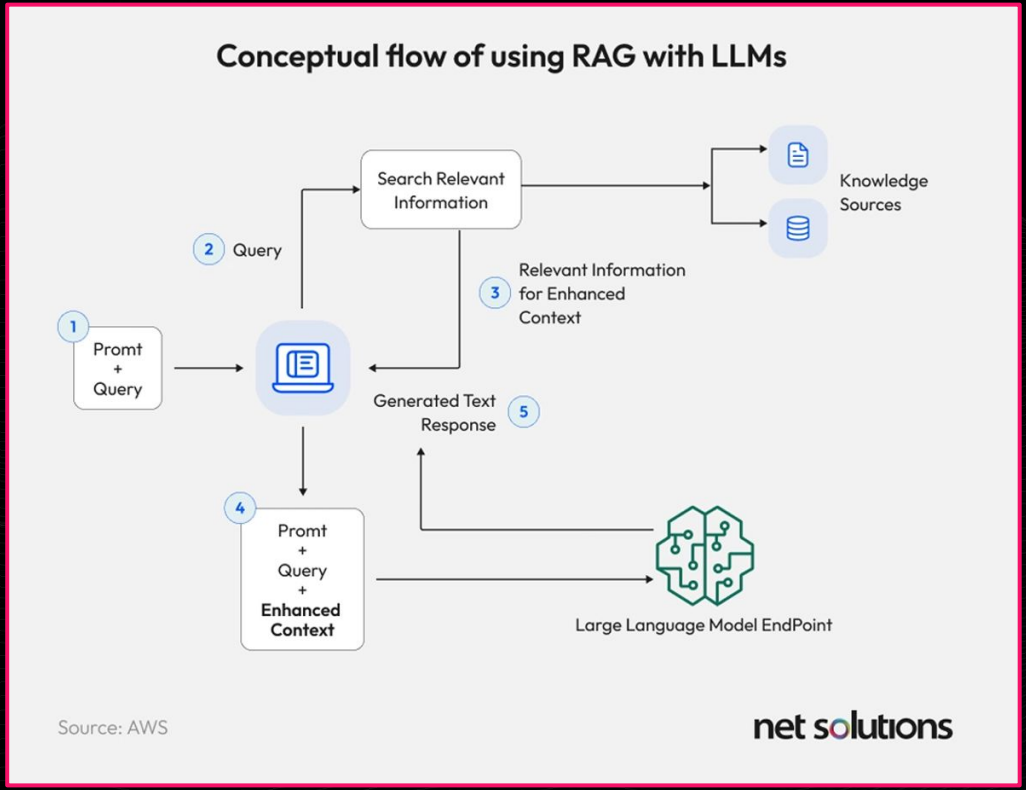
Em extração estruturada, ao invés de uma resposta em texto livre, buscamos que a IA produza dados em um formato padronizado e previsível.



Fonte da imgem: Autor, Nano Banana.

/ RAG

Retrieval-Augmented Generation



/ Function Calling e Output Parsers

/ Function Calling
Chamada de função

Function Calling: Os modelos LLM podem receber definições de funções, com nomes e parâmetros esperados.

Com LangChain, podemos até definir um modelo Pydantic com os campos esperados e obter diretamente uma instância desse modelo do resultado.

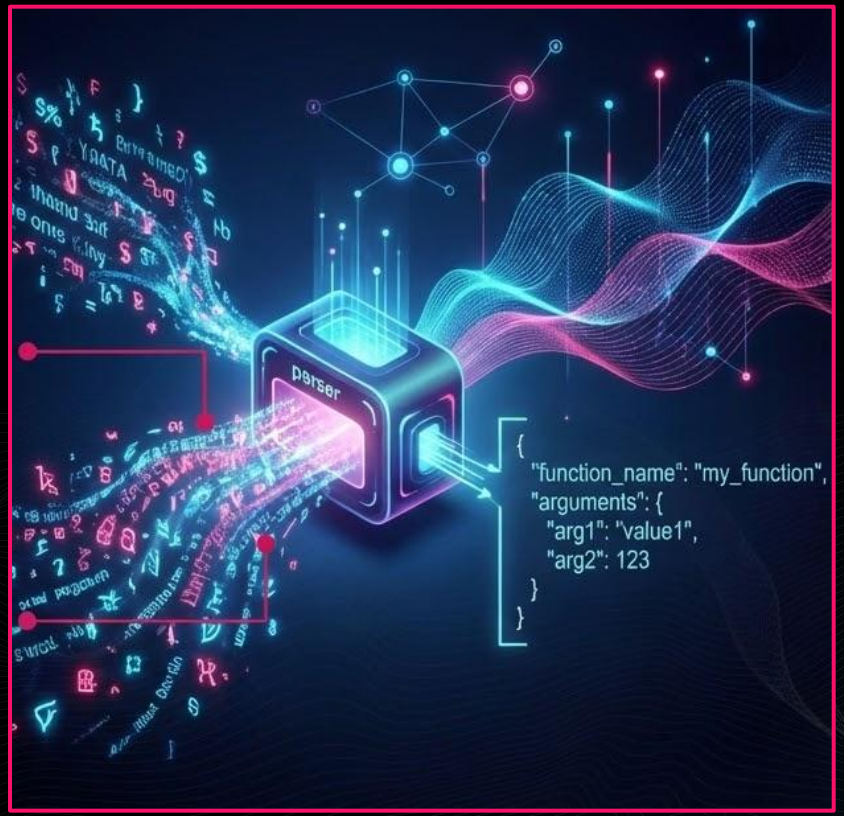
```
{
  "montante": 250000.0,
  "prazo_meses": 18,
  "score_risco": 7
}
```

Fonte da imgem: Autor, Nano Banana.

/ Output Parsers
Estruturando Saídas

Output Parsers: são uma abordagem que o LangChain e outras ferramentas implementam para formatar e interpretar a saída do modelo.

Se perceberem que a consulta pede aquele formato, eles retornam um JSON com argumentos para chamar a função.



Fonte da imgem: Autor, Nano Banana.

/ Validação de Dados com Pydantic

Pydantic: Uma biblioteca Python para validação de dados usando type hints.

Permite definir schemas (modelos de dados) com campos tipados.

O LangChain usa esses schemas para "forçar" o LLM a retornar dados estruturados.

Fonte da imgem: Autor, Nano Banana.

/ Validação de Dados com Pydantic

1. Definimos classes que herdam de `BaseModel` com campos tipados
2. Usamos `Field()` para adicionar descrições que orientam o LLM
3. Podemos usar `Enum` para restringir valores a opções específicas
4. O método `with_structured_output()` do LLM garante que a resposta siga o schema

Vantagens:

✅ Saída sempre no formato esperado (JSON válido)

✅ Tipos validados automaticamente. (int, float, str, etc.)

✅ Campos obrigatórios vs opcionais bem definidos

✅ Fácil exportação para JSON com `model_dump_json()`

/ Hands On

AVALIAÇÃO DE REAÇÃO

POS TECH

FIAP + alura + PM3